

Software Requirements Specification

For ECE1140 Train Management System

Version 0.6

Prepared by Team 3

University of Pittsburgh

2026-09-24

Table of Contents

- [1. Introduction](#)
 - [1.1 Document Purpose](#)
 - [1.2 Product Scope](#)
 - [1.3 Definitions, Acronyms, and Abbreviations](#)
 - [1.4 References](#)
 - [1.5 Document Overview](#)
- [2. Product Overview](#)
 - [2.1 Product Perspective](#)
 - [2.2 Product Functions](#)
 - [2.3 User Characteristics](#)
 - [2.4 Product Constraints](#)
 - [2.5 Assumptions and Dependencies](#)
 - [2.6 Apportioning of Requirements](#)
- [3. Requirements](#)
 - [3.1 External Interfaces](#)
 - [3.2 Functional](#)
 - [3.3 Quality of Service](#)
 - [3.4 Compliance](#)
 - [3.5 Design and Implementation](#)
 - [3.6 AI/ML](#)
- [4. Verification](#)
- [5. Appendixes](#)

Revision History

Name	Date	Reason For Changes	Version
Team 3	2026-08-28	Initial document draft	0.1
Braden	2026-09-21	Restructured requirements into numbered atomic statements	0.2
Braden	2026-09-22	added Communications Interfaces; resolved milestone schedule (3 Work Packages + 3 Iterations)	0.3
Braden	2026-09-23	rebuilt Section 4 as a full requirement-to-verification traceability table	0.4

Name	Date	Reason For Changes	Version
Team	2026-09-24	Applying specificity to some requirements that previously had none. Added UI and UX to definitions table. Changed GUI framework from PyQt to PySide	0.5
Braden	2026-09-24	Renumbered all INTF, FUNC, and DSN requirements to be sequential starting at 001, matching physical document order; updated all cross-references and Section 4 tables accordingly	0.6

1. Introduction

This SRS defines the software requirements for the ECE1140 Train Management System, a simulated rail network developed for the North Shore Extension project contract by PAAC. It covers functional behavior, quality attributes, and verification criteria for the system.

1.1 Document Purpose

This document defines what the ECE1140 Train Management System must do. It is intended for the development team and the customer, the Port Authority of Allegheny County, and serves as the baseline for design and testing throughout the project lifecycle. The document follows the IEEE Standard 830-1998.

1.2 Product Scope

The ECE1140 Train Management System is a Python-based application that provides a train management service with a graphical user interface. It allows operators to dispatch, monitor, and manage trains on a simulated rail network.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
Authority	The maximum distance a train is permitted to travel before stopping
Block	A discrete section of track that can be occupied by at most one train
CTC	Centralized Traffic Control - the operator dispatching interface
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language

Term	Definition
PAAC	Port Authority of Allegheny County
PLC	Programmable Logic Controller — the Boolean-variable program that drives the Track Controller
UX	User Experience

1.4 References

#	Title	Author	Date	Type
1	Project Description and Requirements	PAAC	2026-08	Normative
2	IEEE Std 830-1998: Recommended Practice for SRS	IEEE	1998	Informative
3	UI Style Guide	Team 3	2026	Normative
4	Module Requirements Matrix	Team 3	2026	Informative

1.5 Document Overview

Section 2 provides product context, functions, and constraints. Section 3 contains all verifiable requirements. Section 4 maps each requirement to a verification method. Section 5 contains supporting appendixes. Requirements marked "shall" are mandatory; "should" is recommended; "may" is optional.

2. Product Overview

2.1 Product Perspective

The ECE1140 Train Management System is a new, standalone Python application developed for the ECE1140 course project. It is not a replacement for any existing system. The system simulates a complete commuter rail network and exposes operator, driver, and maintenance interfaces through a graphical UI.

2.2 Product Functions

The system provides a train management service through a graphical UI, including:

- Dispatching and monitoring trains on a simulated network
- Displaying live track and train status to the operator
- Enforcing safe train separation and speed limits
- Simulating train motion, station stops, and failure conditions

2.3 User Characteristics

User Class	Description	Expertise
CTC Dispatcher	Dispatches trains and monitors the network	Moderate technological expertise
Train Driver	Controls a single train in manual mode	Minimal technological expertise; basic operational training
Programmer	Uploads a Boolean PLC program to control the Track Controller module	Superb technological understanding; must write Boolean PLC program
Murphy	Injects failures for testing	Perfect technical understanding. Master at breaking things.
Passenger	Rides the trains	Zero technical knowledge is expected
Track Builder	Builds the track	Mechanical expertise
Port Authority of Allegheny County	The Customer - evaluates the system according the specification they laid out	N/A

2.4 Product Constraints

- The system shall be implemented in Python 3.10 or later
- The system shall run on a standard, modern Windows 11 PC

2.5 Assumptions and Dependencies

#	Assumption / Dependency	Impact if False
A1	Course-provided track layout JSON files are correct and complete	Track simulation would require rework
A2	Lab machines have Python 3.10+ available	Additional setup time required
A3	Module interface contracts are agreed by end of WP1	Late-stage integration failures
A4	PySide6 is installable via pip on all lab machines	GUI framework would need to be swapped or vendored

2.6 Apportioning of Requirements

Functional Area	Iteration 1 (UX)	WP1	Iteration 2	WP2	Iteration 3	WP3 (Final)
Dispatching & Scheduling	—	Basic UI shell	Schedule entry	Auto-dispatch	Full dispatch	Finalized for delivery
Safety & Control	—	Interface contracts	Authority logic	Switch/signal control	Full safety demo	Finalized for delivery
Train & Track Simulation	—	Layout file loading	Physics/occupancy	Failure injection	Complete simulation	Finalized for delivery
Station Operations	—	—	Door logic	Passenger counts	Full station behavior	Finalized for delivery

3. Requirements

Each requirement has a unique, stable identifier (**REQ-<AREA>-<NUM>**), with sub-points numbered **REQ-<AREA>-<NUM>.<SUB>**) used for traceability to Section 4 and to test cases. Numbers are never reused if a requirement is removed.

Area Code	Section	Covers
INTF	3.1 External Interfaces	User, hardware, software, and communications interfaces
FUNC	3.2 Functional	Functional capabilities of the system
NFR	3.3 Quality of Service	Non-functional / quality-of-service requirements
COMP	3.4 Compliance	Course and process compliance requirements
DSN	3.5 Design and Implementation	Design and implementation constraints

3.1 External Interfaces

3.1.1 User Interfaces

REQ-INTF-001: The system shall provide a graphical user interface (GUI) for each defined submodule, including:

- REQ-INTF-001.1: The system shall provide a GUI for the CTC Office submodule.
- REQ-INTF-001.2: The system shall provide a GUI for the Track Model submodule.
- REQ-INTF-001.3: The system shall provide a GUI for the Train Model submodule.
- REQ-INTF-001.4: The system shall provide a GUI for the Track Controller (Hardware) submodule.
- REQ-INTF-001.5: The system shall provide a GUI for the Track Controller (Software) submodule.
- REQ-INTF-001.6: The system shall provide a GUI for the Train Controller (Hardware) submodule.
- REQ-INTF-001.7: The system shall provide a GUI for the Train Controller (Software) submodule.

REQ-INTF-002: The system shall provide a live system map view for the CTC dispatcher, including:

- REQ-INTF-002.1: The system shall display the state of all track blocks on the CTC map view.
- REQ-INTF-002.2: The system shall display the state of all trains on the CTC map view.
- REQ-INTF-002.3: The system shall display the state of all switches on the CTC map view.
- REQ-INTF-002.4: The system shall display the state of all signals on the CTC map view.

REQ-INTF-003: All submodule GUIs shall conform to the UI Style Guide ([documents/UI_Style_Guide.md](#)).

REQ-INTF-004: The system shall present invalid-input error messages using a consistent format across all submodule GUIs.

REQ-INTF-005: All submodule GUIs shall follow a consistent screen-layout convention for navigation, status, and action controls.

3.1.2 Hardware Interfaces

REQ-INTF-006: The system shall provide a separate hardware variation for the Track Controller submodule.

REQ-INTF-007: The system shall provide a separate hardware variation for the Train Controller submodule.

REQ-INTF-008: The system shall accept user input via a standard mouse, keyboard, and monitor.

REQ-INTF-009: The Hardware and Software variations of the Track Controller shall expose equivalent external interfaces.

REQ-INTF-010: The Hardware and Software variations of the Train Controller shall expose equivalent external interfaces.

REQ-INTF-011: The system's GUIs shall be usable on a minimum display resolution of 720x450.

3.1.3 Software Interfaces

REQ-INTF-012: The system shall load track layout data from JSON files at startup.

REQ-INTF-013: All submodules shall share a single simulation clock.

REQ-INTF-014: The system's GUIs shall be implemented using PySide6 as the GUI framework/library.

REQ-INTF-015: The event log produced by REQ-NFR-008 shall be written in CSV format.

REQ-INTF-016: The system may persist information between runtimes outside of the provided track layout details outlined in REQ-INTF-012.

3.1.4 Communications Interfaces

REQ-INTF-017: The system shall implement the track circuit as the communications channel carrying speed and authority data from the Track Controller to the Train Controller.

REQ-INTF-018: The system shall implement a communications channel carrying occupancy and train-presence data between the Track Model and the Track Controller.

REQ-INTF-019: The system shall implement a communications channel carrying speed and authority data from the CTC Office to the Track Controller.

REQ-INTF-020: The system shall implement a communications channel carrying occupancy data from the Track Controller to the CTC Office.

REQ-INTF-021: Each communications channel shall be considered non-vital

3.2 Functional

3.2.1 Simulation

REQ-FUNC-001: The system shall support running the simulation in real-time speed.

REQ-FUNC-002: The system shall support running the simulation in at least 10x wall clock speed.

REQ-FUNC-003: The system shall allow the simulation to be paused.

3.2.2 CTC Office

REQ-FUNC-004: The system shall allow the CTC dispatcher to dispatch a train, including:

- REQ-FUNC-004.1: The system shall allow the CTC dispatcher to enter a destination station for the train being dispatched.
- REQ-FUNC-004.2: The system shall allow the CTC dispatcher to enter an arrival time for the train being dispatched.

REQ-FUNC-005: The system shall determine and send safety-limited speed and authority values to the track controller, including:

- REQ-FUNC-005.1: The system shall determine and send a speed limit to the track controller within safety limits.
- REQ-FUNC-005.2: The system shall determine and send an authority to the track controller within safety limits.

REQ-FUNC-006: The system shall provide a manual dispatch mode for the CTC dispatcher.

REQ-FUNC-007: The system shall provide an automatic dispatch mode for the CTC dispatcher.

REQ-FUNC-008: The system shall allow the CTC dispatcher to switch between manual dispatch mode (REQ-FUNC-006) and automatic dispatch mode (REQ-FUNC-007).

REQ-FUNC-009: While in manual dispatch mode, the CTC dispatcher shall be able to manually dispatch a train to a block.

REQ-FUNC-010: While in automatic dispatch mode, the system shall support loading and running a train schedule.

REQ-FUNC-011: The system shall allow the CTC dispatcher to close a block for maintenance.

REQ-FUNC-012: While in maintenance mode, the CTC dispatcher shall be able to manually set switch positions and send them to the track controller.

REQ-FUNC-013: The system shall allow the CTC dispatcher to monitor train occupancy.

REQ-FUNC-014: The system shall display throughput metrics to the CTC dispatcher, derived from ticket sales data reported by the Track Model (REQ-FUNC-017).

3.2.3 Track Model

REQ-FUNC-015: The system shall load a track model at startup.

REQ-FUNC-016: The system shall display each track segment's properties to the user, including:

- REQ-FUNC-016.1: The system shall display each track segment's grade.
- REQ-FUNC-016.2: The system shall display each track segment's elevation.
- REQ-FUNC-016.3: The system shall display each track segment's length.
- REQ-FUNC-016.4: The system shall display each track segment's speed limit.
- REQ-FUNC-016.5: The system shall display each track segment's direction of travel.
- REQ-FUNC-016.6: The system shall display each track segment's railway crossings.
- REQ-FUNC-016.7: The system shall display each track segment's track heaters.
- REQ-FUNC-016.8: The system shall display each track segment's beacons.

REQ-FUNC-017: The system shall track ticket sales representing passengers waiting at each station.

- REQ-FUNC-017.1: The system shall send ticket sales data to the CTC office.

REQ-FUNC-018: The system shall show passenger movement at each train stop, including:

- REQ-FUNC-018.1: The system shall show the number of passengers boarding each train.
- REQ-FUNC-018.2: The system shall show the number of passengers disembarking each train.

REQ-FUNC-019: The system shall show train occupancy on the track model display.

REQ-FUNC-020: The system shall exchange track circuit signals, including:

- REQ-FUNC-020.1: The system shall send track circuit signals.
- REQ-FUNC-020.2: The system shall receive track circuit signals.

REQ-FUNC-021: The system shall allow the environmental temperature to be set.

REQ-FUNC-022: The system shall simulate track heaters affecting the track environment.

REQ-FUNC-023: The system shall display switch and light state, including:

- REQ-FUNC-023.1: The system shall display switch positions.
- REQ-FUNC-023.2: The system shall display light states.

REQ-FUNC-024: The system shall simulate track failure modes, including:

- REQ-FUNC-024.1: The system shall simulate a broken rail failure mode.
- REQ-FUNC-024.2: The system shall simulate a track circuit failure mode.
- REQ-FUNC-024.3: The system shall simulate a power failure mode.

3.2.4 Train Controller

REQ-FUNC-025: The system shall, in manual mode, regulate train speed at the velocity setpoint received from the train driver.

REQ-FUNC-026: The system shall, in automatic mode, regulate the train speed at the velocity setpoint received from the track controller.

REQ-FUNC-027: The system shall allow the internal train temperature setpoint to be set.

REQ-FUNC-028: The system shall allow the driver to activate the emergency brake.

REQ-FUNC-029: The system shall allow the driver to activate the service brake.

REQ-FUNC-030: The system shall allow the engineer to set the Kp and Ki control gains.

REQ-FUNC-031: The system shall allow the driver to increase and decrease train speed.

REQ-FUNC-032: The system shall use the speed and authority received from the track circuit.

REQ-FUNC-033: The system shall turn train lights on and off.

REQ-FUNC-034: The system shall open and close train doors.

REQ-FUNC-035: The system shall announce stations.

REQ-FUNC-036: The system shall stop the train correctly at each station.

REQ-FUNC-037: The Train Controller shall have a safety-critical architecture, including:

- REQ-FUNC-037.1: The system shall bring a train to a stop when its authority is exhausted.
- REQ-FUNC-037.2: The system shall bring a train to a stop when a failure is detected.

REQ-FUNC-038: The Train Controller shall detect failure conditions reported by the Train Model, including:

- REQ-FUNC-038.1: The system shall detect a train engine failure.
- REQ-FUNC-038.2: The system shall detect a signal pickup failure.
- REQ-FUNC-038.3: The system shall detect a brake failure.

3.2.5 Train Model

REQ-FUNC-039: Given a power command, the system shall calculate train motion using Newton's laws correctly.

REQ-FUNC-040: The system shall display train properties to the user, including:

- REQ-FUNC-040.1: The system shall display train length.
- REQ-FUNC-040.2: The system shall display train height.
- REQ-FUNC-040.3: The system shall display train width.
- REQ-FUNC-040.4: The system shall display train mass.
- REQ-FUNC-040.5: The system shall display train acceleration.

- REQ-FUNC-040.6: The system shall display train velocity.

REQ-FUNC-041: The system shall display crew and passenger count for each train, including:

- REQ-FUNC-041.1: The system shall display crew count for each train.
- REQ-FUNC-041.2: The system shall display passenger count for each train.

REQ-FUNC-042: The system shall receive a track circuit signal containing suggested speed and authority.

REQ-FUNC-043: The Train Model shall track the internal train temperature as commanded by the Train Controller (REQ-FUNC-027).

REQ-FUNC-044: The Train Model shall track the on/off state of train lights as commanded by the Train Controller (REQ-FUNC-033).

REQ-FUNC-045: The Train Model shall track the open/closed state of train doors as commanded by the Train Controller (REQ-FUNC-034).

REQ-FUNC-046: The system shall receive beacon inputs.

REQ-FUNC-047: The system shall allow a passenger to activate the emergency brake.

REQ-FUNC-048: The system shall simulate train failure modes, including:

- REQ-FUNC-048.1: The system shall simulate a train engine failure.
- REQ-FUNC-048.2: The system shall simulate a signal pickup failure.
- REQ-FUNC-048.3: The system shall simulate a brake failure.

3.2.6 Track Controller (Wayside)

REQ-FUNC-049: The wayside track controller shall receive suggested speed and authority from the CTC.

REQ-FUNC-050: The wayside track controller shall load a PLC file.

REQ-FUNC-051: The wayside track controller's PLC language shall be based only on Boolean variables.

REQ-FUNC-052: The wayside track controller shall automatically move switches based on PLC program execution.

REQ-FUNC-053: The wayside track controller shall automatically set traffic light color based on PLC program execution.

REQ-FUNC-054: The wayside track controller shall receive train presence from the track model.

REQ-FUNC-055: The wayside track controller shall send track occupancy to the CTC.

REQ-FUNC-056: The wayside track controller shall activate railway crossing lights and gates.

REQ-FUNC-057: The wayside track controller shall know when the system is in maintenance mode.

REQ-FUNC-058: The Track Controller shall have a safety-critical architecture, including:

- REQ-FUNC-058.1: The system shall limit each train's movement to prevent collisions based on block occupancy.

REQ-FUNC-059: The wayside track controller shall detect failure conditions reported by the Track Model, including:

- REQ-FUNC-059.1: The system shall detect a broken rail failure.
- REQ-FUNC-059.2: The system shall detect a track circuit failure.
- REQ-FUNC-059.3: The system shall detect a power failure.

3.2.7 Moving Block Overlay

Not implemented - out of scope per the customer's requirements.

3.3 Quality of Service

3.3.1 Performance

REQ-NFR-001: The system shall run smoothly in real-time mode, that is:

- REQ-NFR-001.1: The simulation should advance in 1ms increments, or "ticks"
- REQ-NFR-001.2: The simulation shall not drop any ticks during the simulation

REQ-NFR-002: The system shall maintain simulation correctness while running in fast-forward mode.

3.3.2 Security

REQ-NFR-003: The system shall validate all user inputs.

REQ-NFR-004: The system shall reject invalid user input entries without altering simulation state.

3.3.3 Reliability

REQ-NFR-005: The system shall operate without crashes during normal simulation runs.

REQ-NFR-006: The system shall operate without unhandled errors during normal simulation runs.

3.3.4 Availability

REQ-NFR-007: The system shall start up and be ready for use within 20s of launch.

3.3.5 Observability

REQ-NFR-008: The system shall produce a log of significant simulation events for debugging and review.

3.4 Compliance

REQ-COMP-001: The team shall produce UML class diagrams as required by the course rubric.

REQ-COMP-002: The team shall produce UML sequence diagrams as required by the course rubric.

REQ-COMP-003: The team shall produce a UML use case diagram as required by the course rubric.

REQ-COMP-004: All source code and documentation shall be maintained in the course Git repository.

REQ-COMP-005: All commits shall be linked to issue tracker tickets in Jira issue tracking.

REQ-COMP-006: Each team member shall contribute code commits at every milestone.

REQ-COMP-007: The team shall submit a written reflection describing what the team learned from the course.

REQ-COMP-008: The team shall report estimated versus actual hours to complete the project, by team member and in total.

REQ-COMP-009: The team shall report the bug count per week for the last four weeks of development.

REQ-COMP-010: The team shall document the biggest lesson learned by the team.

REQ-COMP-011: The team shall document what the team would do differently if repeating the project.

3.5 Design and Implementation

3.5.1 Installation

REQ-DSN-001: The system shall be launchable as a standalone executable binary.

3.5.2 Build and Delivery

REQ-DSN-002: The repository shall be pre-compiled into an executable binary prior to being delivered to the customer.

3.5.3 Distribution

REQ-DSN-003: The system shall run on a single machine with no external services or network connections required.

3.5.4 Maintainability

REQ-DSN-004: Software modules shall interact through defined interfaces rather than accessing each other's internal state directly.

3.5.5 Reusability

REQ-DSN-005: Common utilities such as the simulation clock and event logger shall be implemented as standalone modules usable by any part of the system.

3.5.6 Portability

REQ-DSN-006: The system shall use platform-neutral file path handling so it runs correctly regardless of the developer's operating system.

3.5.7 Cost

REQ-DSN-007: The team shall produce a cost analysis for the project according to the requirements laid out by the customer in future discussions.

3.5.8 Deadline

Milestone	Date	Deliverables
Iteration 1 — UX Design	2026-09-17	Personas, workflow/information architecture, key inputs & outputs, module UI test
WP1	2026-09-24	SRS, project schedule, coding standards, defect tracking policy, personas; system architecture, UX architecture, UI design, risk assessment
Iteration 2 — Sub-System Presentations	2026-10-08	Sub-system UI, key subsystem inputs/outputs working, separate test UI

Milestone	Date	Deliverables
WP2	2026-10-22	Scrum boards, user stories, architecture & design, test plan
Iteration 3 — System Presentations	2026-11-12	Systemwide integration: dispatch a train from yard to station, sub-systems communicating
WP3 / Iteration 4 — Final Demo	2026-12-10	Complete system: code base, installation manual, test plan results, bug reports, work schedule, demo video

3.5.9 Proof of Concept

REQ-DSN-008: A WP1 POC shall demonstrate a simulated train moving between track blocks under authority control with a basic dispatch UI, validating the core data flow before full implementation begins.

3.5.10 Change Management

REQ-DSN-009: Requirement changes after WP1 shall require full team agreement and shall be recorded in the Revision History table.

3.6 AI/ML

Not applicable. All logic is deterministic and rule-based. No machine learning is used in v1.0.

4. Verification

4.1 External Interfaces

ID	Requirement	Verification Method
REQ-INTF-001	GUI per submodule (7 sub-points)	Demonstration
REQ-INTF-002	CTC live system map view (4 sub-points)	Demonstration
REQ-INTF-003	UI Style Guide conformance	Inspection
REQ-INTF-004	Consistent error message format	Demonstration

ID	Requirement	Verification Method
REQ-INTF-005	Consistent screen-layout convention	Inspection
REQ-INTF-006	Track Controller hardware variation	Inspection
REQ-INTF-007	Train Controller hardware variation	Inspection
REQ-INTF-008	Mouse/keyboard/monitor input	Demonstration
REQ-INTF-009	Track Controller HW/SW interface equivalence	Inspection
REQ-INTF-010	Train Controller HW/SW interface equivalence	Inspection
REQ-INTF-011	Minimum display resolution	Demonstration
REQ-INTF-012	Load track layout from JSON at startup	Test
REQ-INTF-013	Shared simulation clock	Test
REQ-INTF-014	PySide6 as GUI framework	Inspection
REQ-INTF-015	Event log format	Inspection
REQ-INTF-016	State persistence between runs	Test
REQ-INTF-017	Track circuit channel (Track Ctrl -> Train Ctrl)	Test + Demonstration
REQ-INTF-018	Track Model <-> Track Controller occupancy/presence channel	Test + Demonstration
REQ-INTF-019	CTC -> Track Controller speed/authority channel	Test + Demonstration

ID	Requirement	Verification Method
REQ-INTF-020	Track Controller -> CTC occupancy channel	Test + Demonstration

REQ-INTF-021	Vital/non-vital channel designation	Inspection
--------------	-------------------------------------	------------

4.2 Functional

ID	Requirement	Verification Method
REQ-FUNC-001	Real-time speed	Test + Demonstration
REQ-FUNC-002	> = 10x fast-forward speed	Test + Demonstration
REQ-FUNC-003	Pause simulation	Demonstration
REQ-FUNC-004	Dispatch a train w/ destination + arrival time (2 sub-points)	Test + Demonstration
REQ-FUNC-005	Determine/send safety-limited speed + authority (2 sub-points)	Test
REQ-FUNC-006	Manual dispatch mode exists	Demonstration
REQ-FUNC-007	Automatic dispatch mode exists	Demonstration
REQ-FUNC-008	Switch manual/automatic mode	Demonstration
REQ-FUNC-009	Manual dispatch to a block	Test + Demonstration
REQ-FUNC-010	Load and run a train schedule	Test + Demonstration
REQ-FUNC-011	Close a block for maintenance	Demonstration

ID	Requirement	Verification Method
REQ-FUNC-012	Manual switch set in maintenance mode	Demonstration
REQ-FUNC-013	Monitor train occupancy	Demonstration
REQ-FUNC-014	Throughput metrics from ticket sales	Test + Demonstration
REQ-FUNC-015	Load track model at startup	Test
REQ-FUNC-016	Display track segment properties (8 sub-points)	Demonstration
REQ-FUNC-017	Track ticket sales (1 sub-point)	Test
REQ-FUNC-018	Passenger boarding/disembarking (2 sub-points)	Test + Demonstration
REQ-FUNC-019	Track occupancy display	Demonstration
REQ-FUNC-020	Send/receive track circuit signals (2 sub-points)	Test
REQ-FUNC-021	Set environmental temperature	Demonstration
REQ-FUNC-022	Track heaters affect environment	Test
REQ-FUNC-023	Switch/light state display (2 sub-points)	Demonstration
REQ-FUNC-024	Track failure modes (3 sub-points)	Test + Demonstration
REQ-FUNC-025	Manual mode: regulate speed at driver setpoint	Test
REQ-FUNC-026	Automatic mode: regulate speed at track controller setpoint	Test

ID	Requirement	Verification Method
REQ-FUNC-027	Internal temperature setpoint	Demonstration
REQ-FUNC-028	Driver emergency brake	Test + Demonstration
REQ-FUNC-029	Driver service brake	Test + Demonstration
REQ-FUNC-030	Set Kp/Ki control gains	Test
REQ-FUNC-031	Driver increase/decrease speed	Demonstration
REQ-FUNC-032	Use speed/authority from track circuit	Test
REQ-FUNC-033	Train lights on/off	Demonstration
REQ-FUNC-034	Train doors open/close	Demonstration
REQ-FUNC-035	Announce stations	Demonstration
REQ-FUNC-036	Stop correctly at each station	Test + Demonstration
REQ-FUNC-037	Safety-critical architecture (2 sub-points)	Test + Demonstration
REQ-FUNC-038	Detect Train Model failures (3 sub-points)	Test
REQ-FUNC-039	Newton's-laws motion calculation	Test
REQ-FUNC-040	Display train properties (6 sub-points)	Demonstration
REQ-FUNC-041	Display crew/passenger count (2 sub-points)	Demonstration

ID	Requirement	Verification Method
REQ-FUNC-042	Receive track circuit signal	Test
REQ-FUNC-043	Track internal temperature as commanded	Test
REQ-FUNC-044	Track light state as commanded	Test + Demonstration
REQ-FUNC-045	Track door state as commanded	Test + Demonstration
REQ-FUNC-046	Receive beacon inputs	Test
REQ-FUNC-047	Passenger emergency brake	Test + Demonstration
REQ-FUNC-048	Train failure modes (3 sub-points)	Test + Demonstration
REQ-FUNC-049	Receive speed/authority from CTC	Test
REQ-FUNC-050	Load PLC file	Test
REQ-FUNC-051	Boolean-only PLC language	Inspection
REQ-FUNC-052	Auto switch movement via PLC	Test + Demonstration
REQ-FUNC-053	Auto traffic light via PLC	Test + Demonstration
REQ-FUNC-054	Receive train presence from Track Model	Test
REQ-FUNC-055	Send occupancy to CTC	Test
REQ-FUNC-056	Railway crossing lights/gates	Test + Demonstration

ID	Requirement	Verification Method
REQ-FUNC-057	Know when in maintenance mode	Demonstration
REQ-FUNC-058	Safety-critical architecture (1 sub-point)	Test + Demonstration
REQ-FUNC-059	Detect Track Model failures (3 sub-points)	Test

Moving Block Overlay (3.2.7): not implemented, no verification applicable.

4.3 Quality of Service

ID	Requirement	Verification Method
REQ-NFR-001	Smooth real-time operation (2 sub-points)	Test
REQ-NFR-002	Correctness in fast-forward	Test
REQ-NFR-003	Validate user inputs	Test
REQ-NFR-004	Reject invalid input w/o state change	Test
REQ-NFR-005	No crashes	Test
REQ-NFR-006	No unhandled errors	Test
REQ-NFR-007	Startup within 20s	Test
REQ-NFR-008	Event log of significant events	Inspection

4.4 Compliance

ID	Requirement	Verification Method
REQ-COMP-001	UML class diagrams	Inspection
REQ-COMP-002	UML sequence diagrams	Inspection
REQ-COMP-003	UML use case diagram	Inspection
REQ-COMP-004	Source/docs in course Git repo	Inspection
REQ-COMP-005	Commits linked to issue tickets	Inspection
REQ-COMP-006	Commits at every milestone	Inspection

ID	Requirement	Verification Method
REQ-COMP-007	Written course reflection	Inspection
REQ-COMP-008	Estimated vs. actual hours	Inspection
REQ-COMP-009	Weekly bug count	Inspection
REQ-COMP-010	Biggest lesson learned	Inspection
REQ-COMP-011	What the team would do differently	Inspection

4.5 Design and Implementation

ID	Requirement	Verification Method
REQ-DSN-001	Launch as standalone executable binary	Demonstration
REQ-DSN-002	Pre-compiled executable delivered to customer	Inspection
REQ-DSN-003	Single machine, no network	Demonstration
REQ-DSN-004	Modules interact via defined interfaces	Inspection
REQ-DSN-005	Shared clock/logger as standalone modules	Inspection
REQ-DSN-006	Platform-neutral file paths	Inspection
REQ-DSN-007	Cost analysis produced	Inspection
REQ-DSN-008	WP1 POC demo	Demonstration
REQ-DSN-009	Change management process followed	Inspection

5. Appendixes

Appendix A — Track Layout File Format

Track layout files are JSON files provided by the course instructor (see [TrackModel/](#)). Each file lists a line name and an array of blocks; each block includes a block number, section letter, length (m), grade (%), speed limit (km/h), elevation and cumulative elevation (m), and an optional [infrastructure](#) object describing features such as stations, switches, railway crossings, beacons, and transponders.

Appendix B — Safe Braking Distance Formula

$$d = v^2 / (2 \times a)$$

Where d is stopping distance (m), v is current speed (m/s), and a is deceleration (m/s^2).